Week 5

Payment Transaction Management System

Scenario:

An e-commerce company needs to manage customer payment transactions for their orders. The system should store payment details, track payment status, and manage successful, failed, and pending transactions.

The company also wants to analyze different payment methods such as UPI, Card, Net Banking, and Cash. It should generate reports to calculate total revenue, average transaction amount, and identify the most preferred payment method.

Students are required to develop a Payment Transaction Management Module connected with the existing Customer and Order Management modules.

Tables and Attributes:

Customer Table

- CUSTOMER_ID (Primary Key)
- CUSTOMER_NAME
- EMAIL
- PHONE

Orders Table

- ORDER_ID (Primary Key)
- CUSTOMER_ID (Foreign Key)
- ORDER_DATE
- TOTAL_AMOUNT
- ORDER_STATUS

1. Create payment table

The Payment table is created to store customer payment transaction details. It contains six attributes: Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, and Transaction_Amount. Payment_ID is the Primary Key, and Order_ID is the Foreign Key connected to the Orders

table. It helps to record payment methods such as UPI, Card, Net Banking, and Cash. It also tracks successful, failed, and pending transactions.

Payment Table

- PAYMENT_ID (Primary Key)
- ORDER_ID (Foreign Key)
- PAYMENT_DATE
- PAYMENT_MODE
- PAYMENT_STATUS
- TRANSACTION_AMOUNT

```
SQL> CREATE TABLE Payment (
  2      Payment_ID NUMBER PRIMARY KEY,
  3      Order_ID NUMBER,
  4      Payment_Date DATE,
  5      Payment_Mode VARCHAR2(20),
  6      Payment_Status VARCHAR2(20),
  7      Transaction_Amount NUMBER(10,2),
  8      FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
  9  );

Table created.
```

2. Insert values

- Inserted 10 payment transaction records into the Payment table.
- Included different payment modes such as UPI, Credit Card, Debit Card, Net Banking and Cash.
- Stored payment statuses as Success, Pending and Failed.

```
SQL> INSERT INTO Payment VALUES (301, 101, DATE '2026-11-01', 'UPI', 'Succes
s', 55000);

1 row created.

SQL> INSERT INTO Payment VALUES (302, 102, DATE '2026-11-02', 'Credit Card',
 'Success', 1500);

1 row created.

SQL> INSERT INTO Payment VALUES (303, 103, DATE '2026-11-03', 'Debit Card',
'Pending', 25000);

1 row created.

SQL> INSERT INTO Payment VALUES (304, 104, DATE '2026-11-04', 'UPI', 'Succes
s', 12000);

1 row created.

SQL> INSERT INTO Payment VALUES (305, 105, DATE '2026-11-05', 'Net Banking',
 'Success', 5000);

1 row created.

SQL> INSERT INTO Payment VALUES (306, 106, DATE '2026-11-06', 'Cash', 'Pendi
ng', 3000);

1 row created.

SQL> INSERT INTO Payment VALUES (307, 107, DATE '2026-11-07', 'UPI', 'Succes
s', 10000);

1 row created.

SQL> INSERT INTO Payment VALUES (308, 108, DATE '2026-11-08', 'Credit Card',
 'Success', 8000);

1 row created.
```

```
SQL> INSERT INTO Payment VALUES (309, 109, DATE '2026-11-09', 'Debit Card',
'Failed', 6000);

1 row created.

SQL> INSERT INTO Payment VALUES (310, 110, DATE '2026-11-10', 'UPI', 'Succes
s', 4000);

1 row created.

SQL> COMMIT;

Commit complete.
```

3. Display the payment table:

- Displays all payment records stored in the Payment table.
- Shows Payment ID, Order ID, Payment Date, Payment Mode, Payment Status and Transaction Amount.
- Helps to verify and view all customer payment transaction details.

```
SQL> select * from payment;

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       301        101 01-NOV-26 UPI                  Success
             55000

       302        102 02-NOV-26 Credit Card          Success
              1500

       303        103 03-NOV-26 Debit Card           Pending
             25000


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       304        104 04-NOV-26 UPI                  Success
             12000

       305        105 05-NOV-26 Net Banking          Success
              5000

       306        106 06-NOV-26 Cash                 Pending
              3000
```

```
PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       307        107 07-NOV-26 UPI                  Success
             10000

       308        108 08-NOV-26 Credit Card          Success
              8000

       309        109 09-NOV-26 Debit Card           Failed
              6000


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       310        110 10-NOV-26 UPI                  Success
              4000


10 rows selected.
```

4. Manage Successful and Failed Transactions:

Display All Successful Payments

- Displays all successful payment transactions.

- Shows payment details with Success status.
- Helps identify completed customer payments.

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Success';

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- -------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       301        101 01-NOV-26 UPI                  Success
             55000

       302        102 02-NOV-26 Credit Card          Success
              1500

       304        104 04-NOV-26 UPI                  Success
             12000


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- -------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       305        105 05-NOV-26 Net Banking          Success
              5000

       307        107 07-NOV-26 UPI                  Success
             10000

       308        108 08-NOV-26 Credit Card          Success
              8000


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- -------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       310        110 10-NOV-26 UPI                  Success
              4000


7 rows selected.
```

Find Failed Transactions:

- Displays all failed payment transactions.
- Identifies unsuccessful customer payments.
- Helps monitor and track failed transactions.

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Failed';

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
       309        109 09-NOV-26 Debit Card           Failed
              6000
```

Count Total Successful and Failed Payments:

- Counts the total successful and failed payments.
- Groups transactions based on payment status.
- Helps analyze the payment transaction performance.

```
SQL> SELECT Payment_Status, COUNT(*) AS Total FROM Payment WHERE Payment_Sta
tus IN ('Success', 'Failed') GROUP BY Payment_Status;

PAYMENT_STATUS            TOTAL
-------------------- ----------
Failed                         1
Success                        7
```

5. Analyze Payment Methods Used by Customers

Report 1: Payment Mode Analysis:

Number of Transactions Using UPI:

- Counts the total number of UPI transactions.
- Filters records based on UPI payment mode.
- Helps identify UPI usage by customers.

```
SQL> SELECT COUNT(*) AS UPI_Count FROM Payment WHERE Payment_Mode = 'UPI';

 UPI_COUNT
----------
         4
```

Number of Card Payments:

- Counts the total number of credit and debit card payments.
- Filters transactions based on card payment modes.
- Helps analyze customers' card payment usage.

```
SQL> SELECT COUNT(*) AS Card_Count FROM Payment WHERE Payment_Mode IN ('Cred
it Card', 'Debit Card');

CARD_COUNT
----------
         4
```

Most Preferred Payment Method:

- Groups transactions based on payment methods.
- Counts the number of transactions for each payment method.
- Identifies the most preferred payment method among customers.

```
SQL> SELECT Payment_Mode, COUNT(*) AS Total_Transactions FROM Payment GROUP
BY Payment_Mode ORDER BY Total_Transactions DESC;

PAYMENT_MODE         TOTAL_TRANSACTIONS
-------------------- ------------------
UPI                                   4
Credit Card                           2
Debit Card                            2
Net Banking                           1
Cash                                  1
```

Report 2: Revenue Analysis:

Total Revenue Generated:

- Calculates the total revenue generated from successful payments.
- Excludes failed and pending transactions.
- Helps identify the company's total revenue.

```
SQL> SELECT SUM(Transaction_Amount) AS Total_Revenue FROM Payment WHERE Paym
ent_Status = 'Success';

TOTAL_REVENUE
-------------
        95500
```

Revenue by Payment Method:

- Calculates revenue generated by each payment method.
- Groups successful payments based on payment mode.
- Helps compare revenue from different payment methods.

```
SQL> SELECT Payment_Mode,SUM(Transaction_Amount) AS Total_Revenue FROM Payme
nt WHERE Payment_Status = 'Success' GROUP BY Payment_Mode;

PAYMENT_MODE         TOTAL_REVENUE
-------------------- -------------
Credit Card                  9500
Net Banking                  5000
UPI                         81000
```

Average Transaction Amount:

- Calculates the average amount of successful transactions.
- Excludes failed and pending payments.
- Helps analyze the average payment amount made by customers.

```
SQL> SELECT AVG(Transaction_Amount) AS Average_Amount FROM Payment WHERE Pay
ment_Status = 'Success';

AVERAGE_AMOUNT
--------------
    13642.8571
```

Report 3: Customer Payment History:

Display Customer Payment Details

- Displays customer names and their corresponding Order IDs.
- Shows payment mode, transaction amount and payment status.
- Helps maintain and track each customer's payment history.

```
SQL> SELECT C.CUSTOMER_NAME,O.ORDER_ID, P.PAYMENT_MODE,P.TRANSACTION_AMOUNT,
P.PAYMENT_STATUS FROM Customer C JOIN Orders O ON C.CUSTOMER_ID = O.CUSTOMER
_ID JOIN Payment P ON O.ORDER_ID = P.ORDER_ID;

CUSTOMER_NAME                                      ORDER_ID
-------------------------------------------------- ----------
PAYMENT_MODE         TRANSACTION_AMOUNT PAYMENT_STATUS
-------------------- ------------------ --------------------
Arun                                               101
UPI                               55000 Success

Priya                                              102
Credit Card                        1500 Success

Kaviya                                             103
Debit Card                        25000 Pending


CUSTOMER_NAME                                      ORDER_ID
-------------------------------------------------- ----------
PAYMENT_MODE         TRANSACTION_AMOUNT PAYMENT_STATUS
-------------------- ------------------ --------------------
Rahul                                              104
UPI                               12000 Success

Divya                                              105
Net Banking                        5000 Success

Vijay                                              106
Cash                               3000 Pending

CUSTOMER_NAME                                      ORDER_ID
-------------------------------------------------- ----------
PAYMENT_MODE         TRANSACTION_AMOUNT PAYMENT_STATUS
-------------------- ------------------ --------------------
Sneha                                              107
UPI                               10000 Success

Karthik                                            108
Credit Card                        8000 Success

Meena                                              109
Debit Card                         6000 Failed


CUSTOMER_NAME                                      ORDER_ID
-------------------------------------------------- ----------
PAYMENT_MODE         TRANSACTION_AMOUNT PAYMENT_STATUS
-------------------- ------------------ --------------------
Ajay                                               110
UPI                                4000 Success
```

Payment Analysis

The following SQL query is used to analyze successful payment transactions based on payment methods.

- Analyzes the number of successful transactions for each payment method.
- Calculates total revenue and average transaction amount for each payment mode.
- Identifies the payment method that generates the highest revenue.

```
SQL> SELECT  PAYMENT_MODE,COUNT(*) AS TRANSACTION_COUNT,SUM(TRANSACTION_AMOU
NT) AS TOTAL_REVENUE,ROUND(AVG(TRANSACTION_AMOUNT), 2) AS AVG_TRANSACTION FR
OM PAYMENT WHERE PAYMENT_STATUS = 'Success' GROUP BY PAYMENT_MODE ORDER BY T
OTAL_REVENUE DESC;

PAYMENT_MODE         TRANSACTION_COUNT TOTAL_REVENUE AVG_TRANSACTION
-------------------- ----------------- ------------- ---------------
UPI                                  4         81000           20250
Credit Card                          2          9500            4750
Net Banking                          1          5000            5000
```

Conclusion

The Payment Transaction Management System successfully stores and manages customer payment details. It helps track successful, failed, and pending transactions. Payment analysis provides information about revenue, payment methods, and customer payment history. Thus, the system helps the e-commerce company manage payment transactions efficiently and make better business decisions.